

Le switch à 500 €

500 €

Le prix d'un seul switch, introuvable en fin de chantier. Reçu ? Posé ? Perdu ? Personne ne pouvait le prouver — et ce matériel est sous la responsabilité contractuelle de SVT.

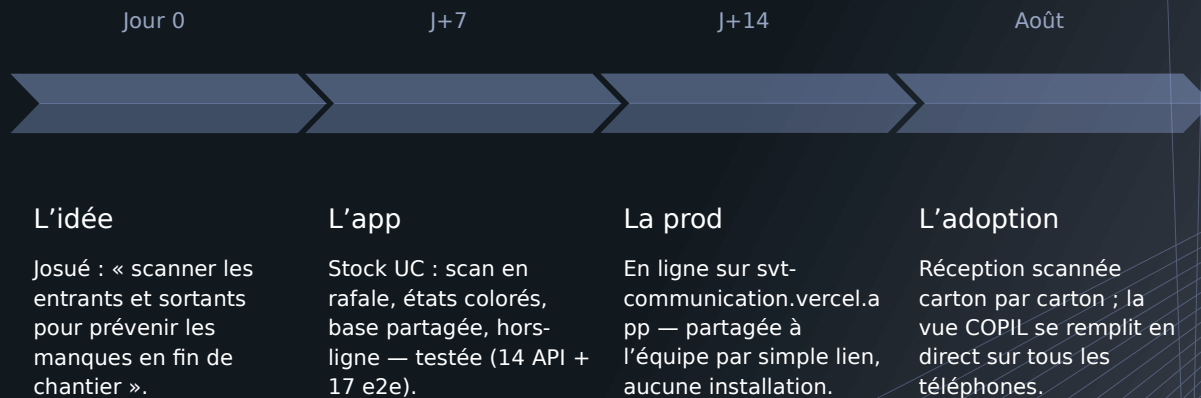
“ La bonne question n’était pas « qui a perdu le switch ? » — c’était « pourquoi notre système permet-il d’en perdre un ? » »

1.

La promesse

« Du changement réel en deux semaines » — la ligne de ma lettre de motivation. La voici, tenue.

Tenue, en quatre étapes



149

équipements suivis en temps réel — capture terrain du 6 août

131

déjà passés en « Programmé » pour le COPIL

0

scan perdu — file hors-ligne, synchronisation automatique

Stock UC : plus jamais un switch sans histoire

Problème

Aucune trace des entrants et sortants : les manques se découvrent en fin de chantier, trop tard. Des cartons identiques, une responsabilité contractuelle exposée.

Action

Une web app zéro installation : caméra, douchette ou saisie, en rafale. Cycle Reçu → Programmé → Planifié → Sorti — chaque mouvement daté, journalisé, partagé.

Appris

Le système doit rendre la perte impossible, pas blâmer les gens : code inconnu = refus net. Et une app de terrain n'a pas le droit de perdre un scan — le hors-ligne est le contrat.

2.

Trois mois, la même méthode

Stock UC n'est pas un coup de chance. Six chantiers, racontés pareil : Problème → Action → Appris.

Six chantiers, **une seule** méthode

Stock UC

Scan & suivi du matériel
billettique — en production,
utilisé par l'équipe.

Usine à rapports

Des preuves terrain aux études
DOCX relues — ~34 600 lignes
de code.

Exploitation

67 932 interventions
cherchables, carte de 899 sites,
COPIL en direct.

Girouettes

Un contrôleur SVT pour
Hanover, Mobitec et Lumiplan
— Hanover opérationnel.

Terrain

Agent piloté par SMS (routeur
RUT241) et banc d'essai
autonome caméra-oracle.

Billettique

MAJ des UC en parallèle,
reconnaissance SSH en lecture
seule.

L'usine à rapports

Problème

Les Études véhicules (clients comme KEOLIS) sont assemblées à la main. « Mauvais rapport → mauvaise info → mauvais équipement » : l'erreur se découvre chez le client.

Action

Un pipeline complet (~34 600 lignes) : preuves → mapping IA → portes de conformité → DOCX. Le document est l'interface ; les planches 3D se rendent sans humain ni GPU.

Appris

Ne jamais croire une IA sur parole : pas de preuve, pas de valeur. Et la langue compte — 363 Mo de rapports validés, minés pour parler celle des relecteurs.

L'exploitation, en direct

Problème

Synchroteam sait « où et qui », pas l'avancement. Les collaborateurs jonglent avec des COxxxx ; le COPIL suit les blocages à la main.

Action

67 932 interventions synchronisées en continu, recherche métier classée, carte site-first de 899 sites, tables business_* lues depuis Google Sheets.

Appris

La règle zéro : on n'écrit jamais dans le système de production. Et écouter la donnée : elle géolocalise le site, donc la carte pense « site » d'abord.

Le soir où le panneau a obéi

Aucune doc, des trames illisibles à 4800 bauds, un checksum inconnu. Puis un glyphe deviné s'affiche sur la girouette : la géométrie était prouvée.



Le même schéma, en plus grand

Problème

Trois familles (Hanover, Mobitec, Lumiplan), trois logiciels vendeurs verrouillés — SVT responsable d'équipements qu'elle ne pilote qu'avec la permission d'un tiers.

Action

Capturer → décoder → rejouer → encapsuler, sur panneau réel. Hanover opérationnel ; Mobitec décodé (jusqu'au test de falsification) ; ~46 000 lignes, carte KiCad v1.

Appris

La méthode vaut plus que l'exploit : un playbook répétable par tout dev SVT. Meilleure validation : un collègue a retrouvé la même table, indépendamment.

Terrain : pas de Wi-Fi, rien d'observable

Problème

Les dépôts ont rarement un réseau fiable. Et le défilement Mobitec n'est produisible par aucun outil vendeur : impossible de sniffer ce qu'on ne peut pas provoquer.

Action

Un bus de commande par SMS sur routeur RUT241 (43 tests verts, listes blanches, simulation par défaut). Et la caméra devient l'oracle : 1 280 expériences autonomes, une nuit.

Appris

La sécurité par défaut n'est pas une option. Quand on ne peut pas observer, on expérimente — une seule variable par essai, reprise sur coupure, tout journalisé.

Chaque boîtier du bus

Problème

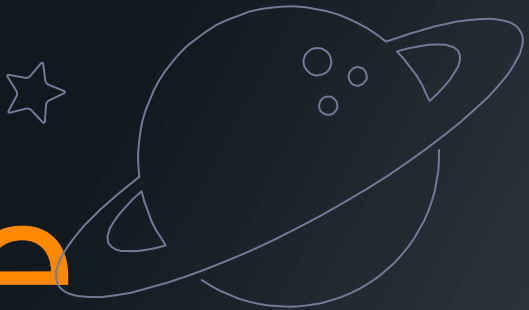
Les mises à jour des UC se font à la clé USB, une unité à la fois. La chaîne embarquée (UC, valideurs, imprimante) reste opaque, sans outillage SVT.

Action

Un disque USB virtuel sur ESP32 ; trois UC mises à jour en parallèle, pilotées depuis un cockpit navigateur.
Reconnaissance de l'UC en SSH, en lecture seule, par playbook.

Appris

Lecture seule d'abord : apprendre un système sans en modifier un octet. L'automatisation vient après la sûreté — et chaque discipline apprise se réutilise ailleurs.



Le drone les sites en 3D



La prochaine histoire : reconstruire chaque site en jumeau 3D par photogrammétrie — mesurer à volonté, rejouer la visite, simuler l'implantation avant de percer le premier trou.

Le drone, en pratique

Problème

Une étude de site ne vit qu'une fois : ce qui n'a pas été mesuré ce jour-là est perdu, et chaque cote oubliée coûte un aller-retour.

Proposition

Un drone photographie le site ; la photogrammétrie reconstruit un jumeau 3D. Notre moteur rend déjà des bus en 3D — et j'ai construit un contrôleur de vol ($PID < 10\text{ ms}$).

Ça ouvre

Des études mesurables, rejouables, archivées ; site + véhicule dans la même scène 3D ; l'implantation simulée avant de percer le premier trou.

Le stage, en six jalons

S1-2 · Stock UC v2 :
listes attendues

1

S5-6 · /rapport en
production

3

S9-10 · Flux & santé
au COPIL

5

S3-4 · Pilote Hanover
de référence

2

S7-8 · Mobitec natif
(capture-diff)

4

S11-12 · Transfert +
essai
photogrammètrie

6

12

dépôts de code livrés pour l'écosystème SVT

95 000+

lignes de code : web, Python, C et C++

3 → 1

outils d'édition vendeurs remplacés par un seul flux SVT

Merci !

Un switch perdu a lancé tout ça. Donnez-moi
trois mois cadrés — voyons ce qu'on construit.

Julien Abdou Gonzales ·

julienabdougonzales@gmail.com ·

abysse8.github.io